

Laboratoire n°5

Temps à disposition : 4 périodes (deux séances de laboratoire)

Objectifs de laboratoire

L'objectif de ce laboratoire est de se familiariser avec les instructions de transfert x86 grâce à l'écriture de fonctions en assembleur x86 et à l'utilisation de macro-instructions. Ce laboratoire permettra aussi de se familiariser avec les pseudo-instructions ainsi que les différents modes d'adressage ARM et x86.

Validation du laboratoire

Un script de rendu vous est fourni afin de packager les fichiers sources modifiés sans inclure tous les fichiers générés (code objet, exécutables, etc.). Ce script s'appelle `rendu.sh` et sera fourni pour tous les laboratoires.

```
reds@reds:~/asm/asm_lab05$ ./rendu.sh
```

Étape 1 - Récupération des sources & environnement Eclipse

```
$ retrieve_lab asm26 lab05
```

Étape 2 - Macros ARM

Dans le laboratoire précédent nous avons implémenté des fonctions de distance. Pour la distance de Manhattan par exemple il fallait calculer la valeur absolue de la différence entre deux valeurs.

Le programme `sliding_puzzle` implémente l'algorithme A^* , une variante de l'algorithme de Dijkstra. Dans le code fourni, l'algorithme A^* utilise une fonction d'évaluation pour estimer à quelle distance une configuration se trouve du but.

L'objectif de cet exercice est d'améliorer la routine d'estimation de la distance écrite en assembleur lors du laboratoire 4.

Comme vu durant le laboratoire 4, la distance de Manhattan entre deux points $A = (x_a, y_a)$ et $B = (x_b, y_b)$, elle se définit comme :

$$D_{man}(A, B) = |x_a - x_b| + |y_a - y_b|$$

Afin d'éviter la duplication de code une option est d'utiliser des fonctions, une autre possibilité est d'utiliser une macro. De même, une macro est utilisée dans le code C.

En l'occurrence dans le fichier « sliding_puzzle.c » on retrouve :

```
#define MAX(X, Y) ((X) > (Y) ? (X) : (Y))
#define MIN(X, Y) ((X) < (Y) ? (X) : (Y))
#define ABSDIFF(A, B) (MAX(A,B) - MIN(A,B))
```

- a) Ajouter dans le fichier « arm/path/student_functions_asm.S » une macro « absdiff » qui prend en paramètre trois registres. La macro doit calculer la valeur absolue de la différence des deux derniers, et placer le résultat dans le 1e registre.
- b) Compléter la fonction « manhattan_distance_asm() » comme dans le laboratoire précédent. Mais cette fois utiliser la macro « absdiff » pour calculer la distance de Manhattan. (vous devez avoir au moins 2 «appel» à la macro absdiff.
- c) Tester le bon fonctionnement de vos fonctions (programmes ARM « puzzle »).
- d) Évidemment, puisque vous allez devoir reprendre du code du laboratoire précédent, il ne sera pas jugé ; seules les macros seront jugées.

Avertissement

Attention aux « data races », en particulier les problèmes d'alias : certains algorithmes peuvent ne plus fonctionner correctement quand deux noms en apparence distincts se réfèrent au même objet.

Étape 3 – Filtre Laplacien (traitement d'image en x86)

Cette étape consiste à implémenter un algorithme de filtrage laplacien sur une image. Le filtre de Laplace est utilisé pour la détection des bords des objets sur une image.

L'idée principale est de détecter la variation de luminosité entre un pixel donné et ses voisins, en appliquant une convolution avec un noyau spécifique.

Principe d'un filtre convoluant

Le filtre de Laplace repose sur la convolution d'une image avec un noyau de transformation. La convolution est une opération qui consiste à appliquer un noyau (ou filtre) à chaque pixel de l'image afin de calculer une nouvelle valeur basée sur les pixels voisins.

Le noyau 3x3 utilisé pour le filtre laplacien est le suivant :

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad (1)$$

Ce noyau met en évidence les transitions en approximant l'inflexion (dérivée seconde) de la luminosité, qui indique entre autres les minimums et maximums locaux.

Convolution

La convolution entre une matrice d'image et un noyau est définie par :

$$I'_{(x,y)} = \sum_{i=-1}^1 \sum_{j=-1}^1 I_{(x+i,y+j)} \cdot K_{(i+1,j+1)} \quad (2)$$

où :

- $I'_{(x,y)}$ est la valeur du pixel après application du filtre,
- $I_{(x+i,y+j)}$ est la valeur d'un pixel voisin dans l'image d'entrée,
- $K_{(i+1,j+1)}$ est la valeur correspondante dans le noyau.

Comme les valeurs résultantes de la convolution peuvent dépasser l'intervalle $[0, 255]$, on applique une normalisation en limitant les valeurs à cet intervalle.

Exemples

Cas d'une image uniforme

Si l'image contient uniquement des pixels de même intensité, par exemple :

$$\begin{bmatrix} 64 & 64 & 64 \\ 64 & 64 & 64 \\ 64 & 64 & 64 \end{bmatrix}$$

Le résultat de la convolution sera nul, car la somme des éléments du noyau est égale à 0



FIGURE 1 – Avant et après application du filtre.

Cas d'un gradient régulier

Considérons un gradient dans une direction particulière :

$$\begin{bmatrix} 0 & 64 & 128 \\ 0 & 64 & 128 \\ 0 & 64 & 128 \end{bmatrix}$$

Le résultat sera encore nul, car les cotés du gradient se compensent exactement.

Cas d'un bord

$$\begin{bmatrix} 64 & 64 & 128 \\ 64 & 64 & 128 \\ 64 & 64 & 128 \end{bmatrix}$$

L'image présente un minimum local dans la direction horizontale, le pixel du centre a donc une valeur plus petite que la moyenne de ses voisins ; le résultat sera donc positif. Avec un maximum local, il serait négatif.

Application sur une Image

L'idée est d'appliquer cette opération sur chaque pixel de l'image et de générer une nouvelle image contenant les résultats de la convolution. Cela produit un effet de détection des bords.

Avant d'implémenter le filtre, il est recommandé d'effectuer des tests sur papier avec différentes valeurs pour bien comprendre l'effet de la convolution et l'impact du noyau utilisé.

Pseudo-code de l'implémentation

```
POUR chaque ligne de l'image FAIRE
  POUR chaque pixel dans la ligne FAIRE
    Accumulateur := 0

    POUR chaque ligne du noyau FAIRE
      POUR chaque élément de la ligne du noyau FAIRE
        Multiplier la valeur de l'élément du noyau
          avec le pixel correspondant
        Additionner le résultat à l'accumulateur
      FIN POUR
    FIN POUR

    SI l'accumulateur < 0 ALORS
      accumulateur := -accumulateur
    SINON SI l'accumulateur > 255 ALORS
      accumulateur := 255
    FIN SI

    Affecter l'accumulateur au pixel correspondant dans l'image destination
  FIN POUR
FIN POUR
```

Avertissement

Attention! Sur les bords de l'image, les voisins du pixel peuvent être hors de l'image, ce qui rend impossible l'application de la convolution. Dans ce cas, il faut simplement copier le pixel original dans l'image de destination.

Les images de départ et de destination sont en niveaux de gris de 0 (noir) 255 (blanc) encodées sur un octet par pixel. Les pixels sont groupés par lignes (ordre "row-major") : l'adresse de départ de l'image pointe sur le pixel [0,0] et l'octet suivant est le prochain pixel sur la ligne et ainsi de suite. Après autant d'octets que la largeur de l'image le pixel suivant sera le premier pixel de la prochaine ligne donc [1,0].

Le kernel de convolution à utiliser se trouve dans le fichier `src/kernels.c` et s'appelle `ridge_detection_3x3`.

Le programme de traitement d'image fourni s'appelle `image_processing` dans le dossier `x86/image_processing/`. Il peut être généré à l'aide du Makefile fourni. Ce programme sera compilé et exécuté sur le machine-hôte (x86_64) et non dans QEMU. Cependant il sera compilé en x86_32 afin de générer de l'assembleur 32-bits comme vu en cours.

- a) L'application `image_processing` nécessite l'installation du paquet « gcc-multilib » (pour les bibliothèques 32-bits) avec la commande suivante :

```
$ sudo apt update
$ sudo apt install gcc-multilib
```

- b) Produire l'exécutable en lançant la commande `make` dans le dossier `x86/image_processing/`. La compilation générera un dossier `build` où l'exécutable s'y trouvera.
- c) Lancer l'application comme ci-dessous :

```
$ cd build/
$ ./image_processing
```

Le programme lit une image fournie en niveaux de gris (par défaut le fichier « `cpu.png` ») et effectue le traitement d'image, c'est-à-dire la réduction de bruit. Deux fichiers seront générés : `expected_result.png` et `student_result.png`.

Le premier fichier `expected_result.png` est le résultat attendu de l'algorithme, écrit en C dans le fichier `image_processing.c`.

Le second fichier `student_result.png` sera le résultat obtenu grâce au code assembleur. L'image sera toute noire au début car la partie assembleur ne fait encore rien pour le moment, le fichier image ne contenant que des "0".

Le programme affiche l'image de départ, le résultat de l'algorithme écrit en assembleur, et si le résultat diffère du résultat attendu, une image avec des pixels rouges pour chaque pixel différent ; cette image est stockée dans `diff.png`.

Il est possible de spécifier un fichier image différent avec l'option `-f` comme suit :

```
$ ./image_processing -f /home/reds/mon_image.jpg
```

i Information

Cette option sera très utile afin de tester l'implémentation avec de petites images. Travailler avec une image de 5x5 pixels sera plus simple pour debugger et trouver des erreurs. Des images plus petites que « `cpu.png` » sont fournies.

Il est possible aussi de demander d'afficher les coordonnées des erreurs ainsi que leurs valeurs avec l'option `-s`, comme suit :

```
$ ./image_processing -s
```

Avec cette option le programme affichera une ligne sur la sortie standard (stdout) pour chaque pixel qui diffère de la solution attendue. Les deux options peuvent être utilisées simultanément. Ceci permet de poser des breakpoints conditionnels (`x = ... y = ...`).

d) Éditer et compléter le fichier assembleur `asm_filter.S`.

i Information

- (a) Les sources du programme `image_processing` sont fournies et peuvent aider à la compréhension. Ne pas hésiter à les modifier au besoin (pour ajouter des `printf()` par ex.). La solution dans le fichier `asm_filter.S` devra toutefois fonctionner avec le programme C d'origine.
- (b) Le fichier assembleur est commenté et pour chaque tâche à effectuer il y a un commentaire "TODO: ...". Une partie du pseudo code vert est déjà présente.
- (c) Dans le fichier assembleur, des commentaires proposent des suggestions.
- (d) La fonction assembleur est appelée par le programme C ; il ne faut en aucun cas changer sa signature ou l'ordre des arguments dans le code assembleur.